Einführung in die Programmierung I
- Richtlinien für Code -

Dr. Andreas Paffenholz
Erik Jansen

TECHNISCHE UNIVERSITÄT DARMSTADT

WiSe 2026/27
13. Oktober 2026

1 Einleitung

Wie bei anderen Texten auch geht es beim Programmieren nicht nur darum, Information in irgendeiner Form korrekt und vollständig zu Papier zu bringen. Eine wesentliche Aufgabe beim Programmieren ist es, das Programm verständlich und klar strukturiert zu schreiben, um anderen die Möglichkeit zu geben, das Programm nachzuvollziehen, um eventuelle syntaktische oder logische Fehler leicht eingrenzen zu können und um das Programm bei Bedarf leicht anpassen oder erweitern zu können. Es wird auch Ihre Zusammenarbeit in den Abgabegruppen erleichtern.

Daher wollen wir in dieser Vorlesung von Anfang an einige Regeln für die Erstellung von C-Programmen vorgeben, an die Sie sich bei den Abgaben ihrer Hausaufgaben halten müssen, und deren Einhaltung in die Bewertung einfließen wird. Diese Regeln sollten Sie nicht als Einschränkung auffassen, sondern sie sollen Ihnen schon von Beginn an an Richtlinien und bewährte Praktiken gewöhnen, die später in größeren Softwareprojekten hilfreich und erforderlich sein werden.

Am Anfang wird es Ihnen vielleicht auch so vorkommen, als ob Ihre Programm durch diese Regeln länger als notwendig oder kompliziert als erforderlich werden. Das wird sich im Verlauf des Kurses sicherlich ändern.

Wir starten mit unseren Regeln auch erstmal mit einer kleinen AUswahl, und werden die Regeln erweitern, sobald wir neue Konzepte in C kennenlernen und neue Regeln sinnvoll und verändlich für Sie sind.

Andererseits gibt es auch Situationen, bei denen nicht jede der folgenden Regeln ist in jeder Situation zwingend oder sinnvoll ist. Hier ist auch Ihre wachsende Erfahrung gefragt, die sinnvolle Anwendung der Regeln einzuschätzen.

Auch in der Vorlesung werden wir häufiger von diesen Regeln abweichen (müssen), da hier noch andere ANnforderungen bestehen, z.B. weil wir den Code kurz halten wollen, damit er noch als ganzes auf den Bildschirm passt.

2 Richtlinien

2.1 Formatierung

Der Quelltext ihres Programms soll einheitlich formatiert werden. Blöcke innerhalb von Funktionen, Schleifen, Verzweigungen werden eingerückt, entsprechend ihrer Tiefe auch mehrfach. Die Einrückungen sollen in Stufen von vier Leerzeichen erfolgen, dabei sollen keine Tabs verwendet werden. Blöcke werden immer in geschweifte Klammern eingefasst, auch wenn der Kontrollblock nur aus einer einzigen Anweisung besteht. In diesem Fall wäre Weglassen syntaktisch in Ordnung, führt aber bei späteren Änderungen leicht zu Fehlern. Dabei soll die öffnende geschweifte Klammer in derselben Zeile wie die zugehörige Anweisung beziehungsweise Funktionsdeklaration stehen.

```c
if (anzahl > 0) {
    verarbeite(items, anzahl);
} else {
    verarbeite_leere_eingabe();
}

void funktion(int param) {
  printf("%d\n", param);
}
```

```c
// richtig:
if (bedingung) {
  rufe_funktion_auf();
}

// falsch
if (bedingung)
    rufe_funktion_auf();
```

Pro Zeile soll grundsätzlich nur eine Anweisung stehen. Die Zeilelänge sollte ungefähr 100 Zeichen nicht überschreiten.

2.2 Namen für Bezeichner (Variablen, Parameter, Konstanten, Funktionen)

Variablennamen sollen möglichst eindeutig beschreiben, welche Bedeutung eine Variable, Funktion oder Konstante hat. Für Variablennamen, die sich aus mehreren Begriffen zusammensetzen, soll `snake_case` verwendet werden. Unnötige Abkürzungen sollten dabei vermieden werden. Namen wie `collatz_laenge` sind verständlicher als `l_collatz` oder `cl`. Anders sieht es hier für die üblichen kurzen Variablen aus, die auch in der Mathematik häufig vorkommen, also z.B. `n` für eine natürliche Zahl (die in der Regel einen variablen Wert oder eine Bereichsgrenze angibt), oder die Indizes `i, j, k`, die auch oft als Schleifenvariablen auftreten.

```c
int collatz_laenge;
double euler_konstante;

int collatz_laenge_berechnen(int startwert);
void ergebnis_ausgeben(int collatz_laenge);
```

```c
for (size_t i = 0; i < n; i++) {
    ...
}
```

Funktionsnamen sollten möglichst klar beschreiben, was die Funktion tut. Typischerweise handelt es sich daher um Verben oder Zusammensetzungen von Verben mit weiteren Erläuterungen. Bei der Verwendung englischer bezeichnungen steht dann das Verb oft vorne. Variablennamen sollten dagegen möglichst beschreiben, welche Information, oder welchen Parameter sie enthalten.

```c
datei_auslesen();
collatz_laenge_bestimmen();
ergebnis_ausgeben();

read_file();
```

```c
int dateigroesse;
int aktueller_index;
int benutzername;
```

Makros und symbolische Konstanten werden in Großbuchstaben geschrieben. In neueren Version von C können auch typisierte Konstanten verwendet werden. Dabei gilt die gleiche Regel.

```c
#define ANZAHL_ITERATIONEN 100
```

```c
enum {
    ANZAHL_ITERATIONEN = 100
};
```

2.3 Variablendeklarationen und Initialisierung

Variablen sollen, soweit möglich, immer in der Nähe ihrer ersten Verwendung deklariert werden und nicht am Anfang einer Funktion oder einer Übersetzungseinheit. Variablen müssen direkt bei der Deklaration initialisiert werden (sofern dem keine syntaktischen Hindernisse im Weg stehen). Sie dürfen nicht uninitialisiert bleiben.

```
int gesamtsumme = 0; // global

for (int i = 0; i < zaehler; i++) {    // lokal
    int quadrat = i*i;                   // lokal
    gesamtsumme += wert[i] * quadrat;
}
```

2.4 Funktionen

Jede Funktion sollte nur eine einzige, klar erkennbare Aufgabe haben. Eine Funktion, die beispielsweise Eingaben liest, diese überprüft, Berechnungen durchführt und anschließend noch eine Ausgabe erzeugt, ist häufig zu umfangreich und sollte auf mehrere Funktionen aufgeteilt werden, z.B.

```
int berechne_collatz_Laenge(int startwert);
void gib_collatz_laenge_aus(int collatz_laenge);
```

Es gibt jedoch keine allgemeingültige maximale Anzahl von Zeilen für eine Funktion. Sehr lange Funktionen sind jedoch häufig ein Hinweis darauf, dass mehrere Aufgaben vermischt wurden und die Funktion auf mehrere Funktionen aufgeteilt werden sollte.

Funktionsprototypen gehören in der Regel in Header-Dateien, während die Implementierung in einer C-Datei erfolgt.

2.5 Datentypen

Datentypen sollen bewusst und passend zu dem Wert, den sie ausdrücken sollen, gewählt werden. Für gewöhnliche ganzzahlige Werte ist `int` häufig ausreichend. Für Größen und Zähler, insbesondere bei der Verwendung mit Standardbibliotheksfunktionen, sollte in der Regel `size_t` verwendet werden.

```
size_t laenge = strlen(text);

for (size_t i = 0; i < laenge; i++) {
    ...
}
```

Für Wahrheitswerte (*booleans*) sollte `short` oder `int` verwendet werden. Dabei sollten nur die Werte `0` und `1` zugewiesen werden. Der Code darf sich jedoch an keiner Stelle darauf verlassen, dass jeder Wahrheitswert immer zu `0` oder `1` auswertet (insbesondere sollten keine Berechnungen damit gemacht werden).

Wenn eine Funktion einen Wert oder ein Objekt nicht verändern soll, soll dies nach Möglichkeit auch im Typ ausgedrückt werden.

```
void name_ausgeben(const char *name);

double summe(const double werte[], size_t anzahl);
```

2.6 Die `main`-Funktion sollte keine inhaltlichen Aufgaben haben

Die Funktion `main` sollte möglichst nur den Ablauf des Programms koordinieren. Die eigentliche Programmlogik gehört in geeignete Funktionen. Dadurch bleibt `main` übersichtlich und die einzelnen Komponenten können leichter getestet und

wiederverwendet werden. Bei `main` muss `void` angegeben werden, wenn keine Kommandozeilenparameter erwartet werden.

```
int main(void) {
    if (!ausfuehren()) {
        return 1;
    }

    return 0;
}
```

2.7 Kommentare

Kommentare sollen erklären, *warum* etwas auf eine bestimmte Weise implementiert wurde, und nicht lediglich wiederholen, was der Code ohnehin bereits aussagt. Links ist ein nutzloses, rechts ein sinnvolleres Beispiel.

Noch

```
// i um 1 erhoehen
i++;
```

```
/* Die erste Zeile enthaelt die
Spaltennamen und wird daher
bei der Verarbeitung uebersprungen.
*/
i++;
```

besser ist es in der Regel, den Code gleich so zu gestalten, dass die Absicht bereits aus dem Code selbst hervorgeht. Statt eines schlecht benannten Werts wie z.B.

```
int x = 3;  // Maximale Anzahl von Versuchen
```

ist beispielsweise eine sprechende Konstante vorzuziehen, wie z.B.

```
enum {
        MAX_ATTEMPTS = 3
};
```

Kommentare sind insbesondere für Algorithmen, Annahmen, nicht offensichtliche Randbedingungen und wichtige Entwurfsentscheidungen sinnvoll.

2.8 Fehlerbehandlung

Wenn eine Funktion mögliche Fehler weitergibt, müssen diese vor der weiteren Bearbeitung im Programm überprüft werden. Ein prominentes Beispiel ist hier das Öffnen von Dateien.

Wenn eine Funktion fehlschlagen kann, sollte nach Möglichkeit schon in der Deklaration erkennbar sein, wie ein Fehler von der Funktion angezeigt wird. `C` stellt hier nicht so viele Möglichkeiten bereit, aber die Rückgabe eines `int`, wenn bei korrekter Ausführung keine Rückgabe erwartet wird, ist ein Indikator. Hier kann der Rückgabewert beispielsweise angeben, ob die Operation erfolgreich war. Rückgabewerte von Funktionen, bei denen ein Fehler auftreten kann, sollen grundsätzlich überprüft werden.

```
FILE *file = fopen(dateiname, "r")
;

if (file == NULL) {
        perror(dateiname);
        return 1;
}
```

```
int schreibe_in_datei(int *
ergebnis);
```

Insbesondere, wenn Funktionen von Bedingungen abhängen, die nicht im Einflussbereich des Programms liegen (wie z.B. Lesen und Schreiben von Dateien, Abfragen von Systemparametern, Reservierung von Speicher), müssen auch ihre eigenen Funktionen mögliche Fehler weitergeben und der Erfolg im Programm überprüft werden.

2.9 Versionsverwaltung

Auch bei kleinen Aufgaben ist die Verwendung eines Versionsverwaltungssystems wie Git sinnvoll. Hier ist z.B. eine typische Struktur für ein kleines Projekt.

```
projekt/
|-- .gitignore
|-- README.md
|-- Makefile
|-- include/
|-- src/
|-- tests/
```

Jeder Commit sollte eine klare und erkennbare einzelne Änderung enthalten und in der Commit-Message auch entsprechend beschrieben sein. Gute Beispiele sind z.B. links, während rechts einige nutzlose Beispiele stehen.

```
        Validierung der Eingabe
hinzugefuegt
        Bildschirmausgabe neu formatiert
```

```
final
final2
wirklich_final
```

Kleine, inhaltlich zusammenhängende Commits erleichtern später die Nachvollziehbarkeit von Änderungen und sind insbesondere bei der Zusammenarbeit in größeren Projekten wichtig.

2.10 Header- und Quelldateien (Erst für spätere Übungen relevant)

Bereits in kleinen Projekten soll zwischen Schnittstelle und Implementierung unterschieden werden. Eine Header-Datei beschreibt dabei die öffentliche Schnittstelle

```
project/
|-- include/
|   |-- collatz.h
|-- src/
|   |-- main.c
|   |-- collatz.c
|-- Makefile
```

```c
#ifndef COLLATZ_H
#define COLLATZ_H

int collatz_laenge_berechnen(int startwert);
int collatz_laenge_ausgeben(int collatz_laenge
    );

#endif
```

Die Implementierung der deklarierten Funktionen befindet sich in der zugehörigen C-Datei. Header-Dateien müssen mit Include Guards gegen mehrfaches Einbinden geschützt werden. Nur in Ausnahmefällen sollen Funktionen direkt in Header-Dateien implementiert werden.

2.11 Include-Anweisungen (Erst für spätere Übungen relevant)

Es sollen nur Header-Dateien eingebunden werden, die tatsächlich benötigt werden. Eine sinnvolle Reihenfolge ist dabei

1. der zugehörige eigene Header,
2. Header der C-Standardbibliothek,

3. Header externer Bibliotheken,
4. weitere projektspezifische Header.

Dass der zugehörige Header einer Quelldatei zuerst eingebunden wird, hilft unter anderem dabei, fehlende Deklarationen im Header frühzeitig zu erkennen.

2.12 Magic Numbers vermeiden (Erst für spätere Übungen relevant)

Unverständliche oder mehrfach verwendete Zahlenwerte sollen nicht, wie im linken Beispiel, direkt im Quelltext stehen, sondern wie rechts als Konstanten eingeführt werden.

```
if (anzahl_versuche > 3) {
    ...
}
```

```
enum {
    MAX_ANZAHL_VERSUCHE = 3
};
if (anzahl_versuche > MAX_ANZAHL_VERSUCHE) {
    ...
}
```

Hiervon gibt es sicherlich Ausnahmen. Werte wie `0` oder `1` sind in vielen Zusammenhängen unmittelbar verständlich. Entscheidend ist, dass inhaltilich relevante Konstanten eine verständliche Bezeichnung erhalten.

2.13 Zeiger (Erst für spätere Übungen relevant)

Für Zeigerdeklarationen soll die Schreibweise

```
int *value;
const char *text;
```

verwendet werden. Es sollte immer klar sein, wer das Objekt, auf das der Zeiger zeigt, besitzt, und wer für dessen Lebensdauer beziehungsweise Freigabe verantwortlich ist.

2.14 Dynamischer Speicher (Erst für spätere Übungen relevant)

Die Anforderung und Freigabe von Speicher sollte wie folgt aussehen:

```
int *values = malloc(count * sizeof(*values));

if (values == NULL) {
    /* Fehler behandeln */
    return false;
}

/* values verwenden */

free(values);
```

Bei `malloc` sollte nach Möglichkeit die Größe des Objekts über den Ausdruck `sizeof(*pointer)` bestimmt werden, also

```
malloc(count * sizeof(*values))
```

Das vermeidet unnötige Wiederholungen des Typs und bleibt auch dann korrekt, wenn sich der Typ des Zeigers später ändert.

Nach `free` darf auf das freigegebene Objekt nicht mehr zugegriffen werden. Falls der Zeiger anschließend noch im Gültigkeitsbereich bleibt und eine weitere Verwendung möglich ist, kann er auf `NULL` gesetzt werden.

```
free(values);
values = NULL;
```

Das Setzen auf `NULL` ist jedoch kein Ersatz für ein korrektes Speicherverwaltungskonzept. Entscheidend ist, dass Objekte genau entsprechend ihrer Lebensdauer verwendet und freigegeben werden.

2.15 Problematische syntaktische Konstruktionen (Erst für spätere Übungen relevant)

Manche Konstruktionen der Sprache `C` und der Standardbibliothek sind besonders fehleranfällig oder veraltet und sollten nicht verwendet werden. Das betrifft insbesondere

- globale veränderliche Variablen,
- ungeprüfte Rückgabewerte von `scanf()` und ähnlichen Funktionen,
- Eingabeverarbeitung ohne ausreichende Begrenzung der Puffergröße,
- unerklärte oder unnötige Typumwandlungen,
- Grundsätzlich nicht zu verwenden sind
  - `gets()`
  - `goto` für gewöhnliche Kontrollflüsse,
  - compilerabhängige Erweiterungen (insbesondere die GNU compiler extensions)
- Konstruktionen, deren Verhalten von impliziten und möglicherweise verlustbehafteten Konvertierungen abhängt.

3 Abschließende Bemerkung

Die Regeln dieses Dokuments sind kein Selbstzweck. Ein guter Programmierstil soll vor allem dabei helfen, Programme zu schreiben, die auch nach längerer Zeit noch verstanden und verändert werden können.

Bei kleinen Übungsprogrammen erscheint manche Regel möglicherweise übertrieben. In einem größeren Projekt wird jedoch schnell deutlich, warum einheitliche Formatierung, aussagekräftige Namen, klare Schnittstellen, kontrollierte Speicherverwaltung und konsequente Fehlerbehandlung wichtig sind.

Die im Kurs verwendeten Regeln sollen deshalb von Anfang an als Handlungsvorgabe für das Schreiben größerer Programme verstanden werden. Nicht jede Aufgabe wird alle Aspekte gleichzeitig benötigen.

Quelltext wird nicht nur für den Compiler geschrieben, sondern vor allem auch für Menschen, die ihn später lesen, verstehen und ändern müssen.